

Wireshark

7CSEF006W

Dr Ayman El Hajjar

Academic Year: 2025/26

Install Wireshark

➡ In this lab, you are free to use Wireshark on your machine or set up a virtual machine to use Wireshark inside.

↳ Wireshark is **already installed** on **Kali Linux**.

↳ To download Wireshark, visit [wireshark website](#).

1 Start with Wireshark

1.1 Objectives

When you complete this section, you will be able to:

- ➡ Launch and configure Wireshark network packet analysis software.
- ➡ Capture raw network traffic.
- ➡ Apply and use filters, including building expressions.

1.2 What is Wireshark

- ➡ Wireshark is a network analyser software — it is also called a Network monitor or packet sniffer.
- ➡ Used to monitor and troubleshoot network traffic.
- ➡ Captures raw bits, decodes them to a specification, and presents them for use and interpretation.
- ➡ Useful for analysis of network problems, detecting network intrusions, monitoring bandwidth utilisation, etc.
- ➡ There are many other applications, however, Wireshark is popular, open-source, and has a wealth of online resources and support.

Wireshark interface

- ➡ Before you continue below, please spend some time exploring the Wireshark interface.
- ➡ Look specifically at the items listed below:
 - ↳ What are the menu interface items.
 - ↳ Capture view.
- ➡ Go through "Chapter 1. Getting Started with Wireshark – What, Why, and How?" from [Wireshark Network Security book](#) to understand Wireshark and its purpose.

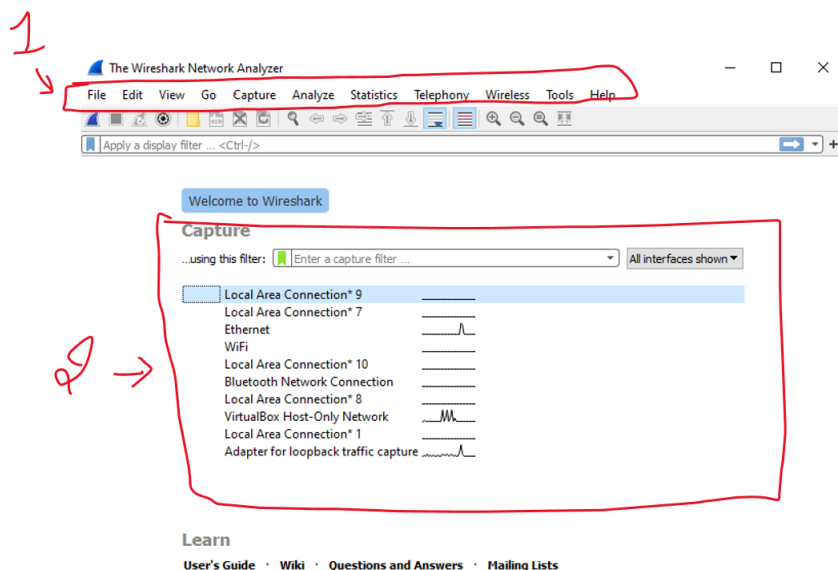


Figure 1.1: Getting started - Wireshark interface

2 Examining TCP/IP Layers

In this section, we will investigate different protocols from four layers of the TCP/IP model.

2.1 TCP Example

- ➡ Open the TCP_cap.pcap file by either double-clicking on it or from inside Wireshark.
- ➡ Identify the IP addresses of the TCP initiator and receiver.
- ➡ Can you identify the three-way handshake of a TCP connection?

2.2 IPv4 - Traceroute

- ➡ Traceroute enables a user to display the route a packet takes across an IP network, as well as measuring and displaying the transit delays involved.
- ➡ Typically, three packets are sent with the round-trip time of each packet recorded for each successive node.
- ➡ The sum of the average times taken across the packets for each hop provides a measure of the total time required to establish the connection.
- ➡ If all three packets are lost more than twice, traceroute considers the connection lost and the route therefore cannot be determined.
- ➡ The time-to-live (TTL), also known as hop limit, is gradually increased with each successive packet.
- ➡ Each router decrements this value by one, discarding those with a TTL value of zero and returning an ICMP Time Exceeded error.
- ➡ The Internet Control Message Protocol (ICMP) is used by network devices to send error messages and operational information.
- ➡ Traceroute uses the Time Exceeded messages to identify the routers involved in the transmission of packets until the final destination is reached, which returns an ICMP Echo Reply.
- ➡ Traceroute can also use TCP instead of ICMP. This is commonly used to bypass firewall filters.
- ➡ In this section, we are capturing traffic for a traceroute to **www.google.co.uk**.
- ➡ While capturing traffic:
 - ↳ If you are on Windows, open the command line and type:

```
> tracert www.google.co.uk
```
 - ↳ If you are on Linux, open the terminal and type:

```
> traceroute www.google.co.uk
```
 - ↳ If you are on Linux, you can also use TCP-based traceroute:

```
> traceroute -T www.google.co.uk
```
- ➡ A good website to understand traceroute and Wireshark: [Hacking Articles - Working with Traceroute using Wireshark](#).
- ➡ Now that we have captured traffic for a traceroute to Google servers, think about your network infrastructure and the type of hops between your device and Google servers. Try to identify the various devices on the route to Google.

2.3 HTTP Conversation

➡ In this section, we will look at analysing the HTTP header and inspecting the HTTP packet structure.

➡ HTTP is an application layer protocol. It works by using two different types of request messages between client and server:

↳ Request Line (**GET** or **POST**).

↳ **GET** request messages ask the server to download contents/files from a website or service.

↳ **POST** request messages upload data to the server.

➡ In the request message, you should see the following information:

↳ Request Line (**GET** or **POST**).

↳ Uniform Resource Identifier (URI).

↳ Information such as details about the client making the request.

↳ Status codes:

↳ 2xx Success, e.g. **200 OK**.

↳ 3xx Redirection — further action required, e.g. **301 Moved Permanently**.

↳ 4xx Client Error — request cannot be fulfilled, e.g. **404 Not Found**.

↳ 5xx Server Error — server failed to fulfil request, e.g. **502 Bad Gateway**.

↳ Header fields, e.g. type of content.

↳ Data (if included in the request).

➡ Open the `tcp_cap` file in Wireshark.

➡ Apply a `http` filter.

➡ Analyse the HTTP conversation.

➡ What can you deduce from the HTTP fields?

➡ Now let's capture an insecure HTTP login page:

↳ While running Wireshark and capturing traffic, open a browser and visit:

<http://www.stealmylogin.com/demo.html>

↳ Follow the instructions on the page and log in.

↳ After logging in, stop Wireshark from capturing.

- ↳ Apply the HTTP filter again and look for HTTP packets.
- ↳ Look for **POST** requests — since the username and password you entered were sent to the server via POST.
- ↳ In the packet contents, click on **HTML Form URL Encoded**. You should be able to see the credentials you used on the website — they were sent in plaintext.

2.4 UDP DNS Traffic

- ➞ In this section, you will be able to describe the main content of the UDP header.
- ➞ You will also inspect the IP packet structure.
- ➞ Identify and describe DNS queries (IPv4, IPv6, Mail).
- ➞ Conduct a manual name resolution.
- ➞ Domain Name System (DNS) is an application layer protocol:
 - ↳ Primarily uses UDP.
 - ↳ Uses TCP for full zone transfers.
 - ↳ Uses port **53**.
 - ↳ Request messages are either **Query** (from client) or **Response** (from server).
- ➞ Open the `udp_cap` file in Wireshark.
- ➞ Apply a `udp` filter.
 - ↳ Inspect frame **13**.
 - ↳ Look specifically at the UDP details.
- ➞ Now apply a `DNS` filter.
 - ↳ Inspect frames **1017** and **1032**.
 - ↳ Look closely at UDP and DNS details.
- ➞ What information were you able to obtain from those two steps?

2.5 RTP Traffic

- ➞ The Real-time Transport Protocol (RTP) is a network protocol for delivering audio and video over IP networks.
- ➞ RTP is used in communication and entertainment systems that involve streaming media, such as telephony, video teleconference applications including WebRTC, television

services, and web-based push-to-talk features.

- ➡ Open the `rtp_cap` file in Wireshark.
- ➡ Apply an `rtp` filter.
 - ↳ Inspect the RTP subtree packets.
 - ↳ Now click on **Telephony** in the top menu and select **RTP** then **RTP Streams**.
 - ↳ Select all the streams identified and click on **Analyse**.
 - ↳ Can you decode the stream?

2.6 FTP Traffic

- ➡ The File Transfer Protocol (FTP) is a standard network protocol in the application layer. It is used for the transfer of computer files between a client and server on a computer network.
- ➡ In this section, you have a scenario where an employee has reportedly been downloading files via FTP that are believed to be non-work related and potentially malicious.
- ➡ Open the `ftp_cap` file in Wireshark.
- ➡ Apply an `ftp` filter.
 - ↳ Timing of the events.
 - ↳ Equipment used in communication, including:
 - ↳ Manufacturer
 - ↳ IP Address
 - ↳ Port(s) used
 - ↳ Nature of the communication, including:
 - ↳ Which FTP commands were issued and their function.
 - ↳ Details of files copied to the local machine.
 - ↳ Concluding summary, including:
 - ↳ Which actions the user took and why.
 - ↳ The reconstructed contents of any files copied.

2.7 A Cyber Incident Traffic

- ➡ You have been asked to analyse a trace of network traffic affecting a piece of your equipment, then provide a summary of what was occurring within the traffic.
- ➡ Open the `incident_cap` file in Wireshark.
- ➡ Explore the captured traffic and try to deduce what is happening.
- ➡ Write down the following information:
 - ↳ Timing of the events.
 - ↳ Equipment used in communication, including:
 - ↳ Manufacturer
 - ↳ IP Address
 - ↳ Port(s) used
 - ↳ Geo-location and any ambiguity
 - ↳ Who initiated communications
 - ↳ Nature of the communication, including:
 - ↳ The protocols used
 - ↳ What was occurring within each protocol
 - ↳ Screenshots to support statements
 - ↳ Concluding summary, including:
 - ↳ What took place
 - ↳ What, if any, action should be taken